

UNIVERSIDADE DO ESTADO DE SANTA CATARINA – UDESC

CENTRO DE CIÊNCIAS TECNOLÓGICAS – CCT

CURSO DE CIÊNCIA DA COMPUTAÇÃO

PROJETO FINAL: SISTEMA DE GESTÃO DE ASSINATURAS

INTEGRANTES:

KELWIN EFRAIN BAGNHUK DA SILVA

MATHEUS EDUARDO GOMES DE SANTANA

PROFESSOR: REBECA SCHROEDER FREITAS

DISCIPLINA: BANCO DE DADOS I

JOINVILLE

2026

SUMÁRIO

1. INTRODUÇÃO.....	3
2. REGRAS DE NEGÓCIO.....	3
3. DIAGRAMA ENTIDADE RELACIONAMENTO.....	4
3.1. Restrições de Integridade.....	4
3.2. Tradução para o Projeto Lógico.....	5
4. INSTRUÇÕES DE COMPILAÇÃO.....	6
4.1. Requisitos do Sistema.....	6
4.2. Preparação do Banco de Dados.....	6
4.3. Configuração do Projeto (NetBeans).....	6
4.4. Configuração de Conexão e Execução.....	7

1. INTRODUÇÃO

Este trabalho tem como objetivo apresentar a modelagem e implementação de um banco de dados relacional para um Sistema de Gestão de Assinaturas. O projeto foi desenvolvido para a disciplina de Banco de Dados I, aplicando desde a modelagem conceitual até a criação física das tabelas no SGBD PostgreSQL.

O sistema foi programado em Java com a API JDBC. Para cumprir as regras do trabalho, não foram utilizados frameworks, garantindo que as operações de manipulação (inserir, alterar, remover e listar) e as consultas complexas fossem feitas diretamente por comandos SQL.

2. REGRAS DE NEGÓCIO

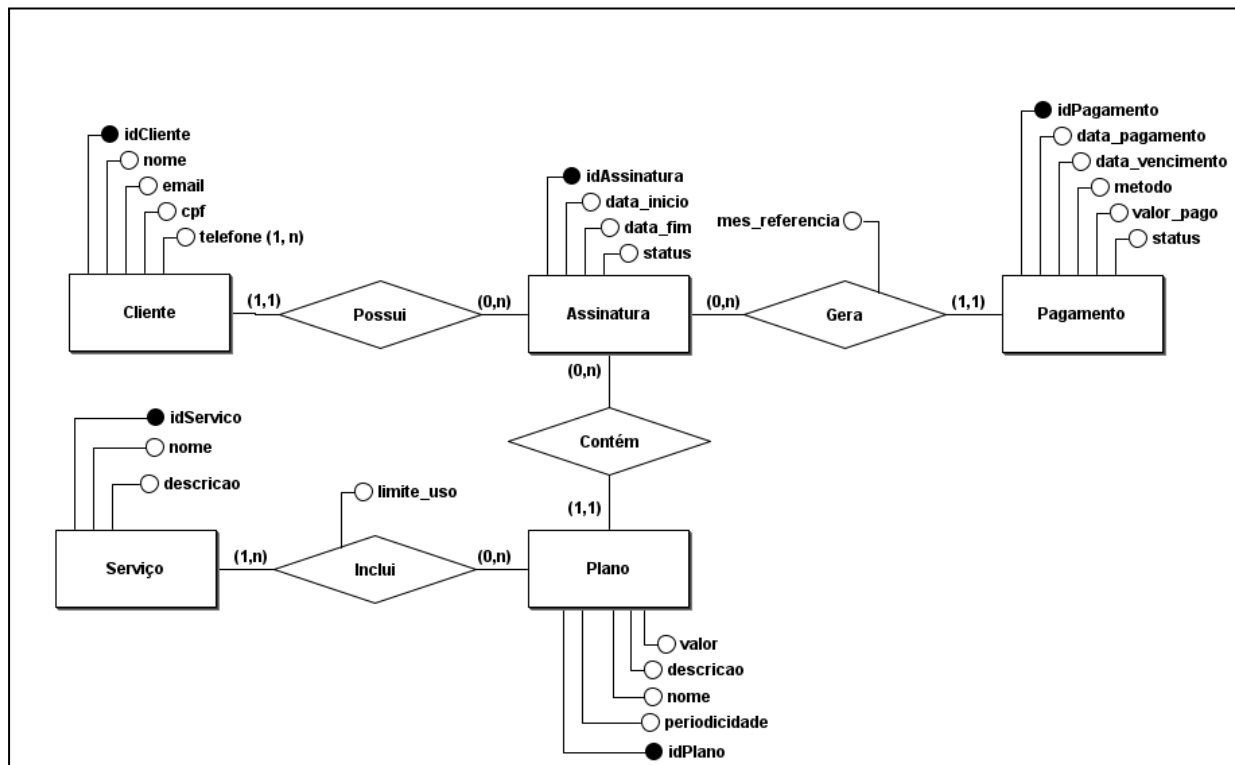
O sistema gerencia planos de assinatura e seus pagamentos. As regras principais são:

- O sistema possui **Planos** (ex: Básico, Premium) com um valor e uma periodicidade de cobrança (mensal, anual, etc.).
- Cada plano inclui um ou vários **Serviços** (ex: Streaming, Nuvem). Um mesmo serviço pode fazer parte de vários planos diferentes.
- Os **Clientes** contratam planos gerando **Assinaturas**. Uma assinatura liga um cliente a um único plano, registrando a data de início, data de fim (que pode ser vazia/nula) e o status.
- Cada assinatura gera **Pagamentos** periódicos, que guardam o valor pago, o método de pagamento, as datas e o status (pago, pendente, atrasado).

3. DIAGRAMA ENTIDADE RELACIONAMENTO

A modelagem conceitual gerou 5 entidades principais

Figura 1 – Diagrama



Fonte: Elaborado pelos autores.

3.1. Restrições de Integridade

Conforme a teoria da modelagem de dados, o diagrama Entidade-Relacionamento nem sempre consegue expressar graficamente todas as restrições de um domínio de aplicação. Para garantir a semântica correta do Sistema de Gestão de Assinaturas, definem-se as seguintes restrições de integridade em relação ao modelo conceitual:

R.I 01: O atributo *periodicidade* da entidade *Plano* é restrito aos valores "mensal", "trimestral", "semestral" ou "anual".

R.I 02: O atributo *status* da entidade *Pagamento* é restrito aos valores "pago", "pendente" ou "atrasado".

R.I 03: O atributo metodo da entidade Pagamento é restrito aos valores "cartao_credito", "boleto", "pix" ou "debito_automatico".

R.I 04: O atributo status da entidade Assinatura é restrito aos valores "ativa", "cancelada" ou "suspensa".

R.I 05 (Opcionalidade): Como uma assinatura pode ser vitalícia, o atributo “data_fim” é opcional (pode receber nulo). Da mesma forma, como um pagamento pendente ainda não ocorreu, o atributo “data_pagamento” também é opcional e, no caso do “limite_uso” também poderá ser opcional, assumindo o valor “ilimitado” caso não seja preenchido.

3.2. Tradução para o Projeto Lógico

Abaixo está o esquema resultante do mapeamento lógico (onde # indica Chave Primária, & indica Chave Estrangeira e [] indica atributos opcionais):

- Cliente (#id_cliente, nome, email, cpf, telefone)
- Plano (#id_plano, nome, descricao, valor, periodicidade)
- Servico (#id_servico, nome, descricao)
- Assinatura (#id_assinatura, data_inicio, [data_fim], status, &id_cliente, &id_plano)
- Pagamento (#id_pagamento, data_vencimento, [data_pagamento], mes_referencia, metodo, valor_pago, status, &id_assinatura)
- Plano_Servico (#&id_plano, #&id_servico, [limite_uso])

4. INSTRUÇÕES DE COMPILAÇÃO

Esta seção descreve os requisitos e o passo a passo para configurar, compilar e executar o sistema localmente.

4.1. Requisitos do Sistema

- **Linguagem:** Java Development Kit (JDK) 8 ou superior.
- **Banco de Dados:** PostgreSQL (rodando localmente na porta padrão 5432).
- **Driver:** Driver JDBC do PostgreSQL localizado na pasta lib/.
- **IDE Recomendada:** Apache NetBeans (utilizando o padrão de projeto *Java with Ant*).

4.2. Preparação do Banco de Dados

1. Abra o SGBD (ex: pgAdmin) e crie um banco de dados em branco chamado subscriptions.
2. Abra a ferramenta de *Query* e execute todo o conteúdo do arquivo schema.sql fornecido. Isso criará a estrutura física das 6 tabelas com suas respectivas chaves e restrições de integridade.

4.3. Configuração do Projeto (NetBeans)

1. No NetBeans, crie um novo projeto acessando **Arquivo > Novo Projeto > Java with Ant > Aplicação Java**. Nomeie o projeto (desmarque a opção "Criar Classe Principal").
2. No explorador de arquivos do sistema operacional, substitua a pasta src/ vazia gerada pelo NetBeans pela pasta src/ do projeto, contendo os pacotes organizados (bean, controller, model, conexao) e a classe Principal.java.
3. **Adição do Driver:** No NetBeans, clique com o botão direito na pasta **Bibliotecas** do projeto, selecione **Adicionar JAR/Pasta** e aponte para o arquivo *postgresql-42.x.x.jar*.

4.4. Configuração de Conexão e Execução

1. Acesse o arquivo `conexao/Conexao.java` e certifique-se de que a variável `senha` contém a credencial correta do usuário postgres da sua máquina local.
2. Para iniciar o sistema, clique com o botão direito no arquivo **Principal.java** e selecione **Executar Arquivo**. A interação com o sistema (inserir, remover, listar) ocorre diretamente no terminal da IDE.